

ECE3300

Spring 2026

Final Project - Reaction Time Game

Kayla Tojin, Lupe Becerra, Rodrigo Vallejo, Justin Sov

5/11/2026

1.1 System Description

The Reaction Time Game is a real-time digital system implemented on the Nexys A7 FPGA board. The objective of the design is to measure human reaction speed by presenting randomized directional prompts and requiring the user to respond using push-button inputs within a specified timing window. The system integrates multiple hardware modules to form a complete embedded architecture. These include a finite state machine (FSM) for game control, a VGA display subsystem for graphical output, a UART subsystem for serial logging, and a seven-segment display driver for numerical visualization. Input reliability is ensured through debounce circuits applied to all push-button signals.

The system operates in repeating rounds, where each round follows a consistent sequence:

1. The user selects a difficulty mode using the switches.
2. The game begins by toggling SW[0] from high to low, which resets and initializes the system.
3. A random directional target is generated.
4. A randomized delay period occurs (WAIT state).
5. The system enters an active response phase (GO state). The VGA interface displays a graphical button pad representing the Nexys board buttons and highlights a random directional arrow or center button in green, where the player must press the corresponding physical button within the selected time limit. Simultaneously, the seven segment display begins counting upwards in milliseconds to measure the user's reaction time.
6. User input is evaluated for correctness and timing.
7. The system transitions to win, lose, or next round logic.

This structure enables consistent timing evaluation while maintaining unpredictability in user interaction.

1.2 Required I/Os

The system is synchronized using a 100 MHz system clock, which drives all sequential logic. A set of switches provides configuration control, while push-buttons act as the main user input interface.

1.2.1 Inputs

Clock - Global system clock.

Switches (SW[15:0]) - Configuration for difficulty mode selection by adjusting the timing

constraints for the allowed reaction time. Each switch within each mode decreases the allowed response time by two hundred milliseconds from right to left across the board.

SW[15:11] - Easy Mode (3300ms, 3100ms, 2900ms, 2500ms)

SW[10:6] - Normal Mode (2300ms, 2100ms, 1900ms, 1700ms, 1500ms)

SW[5-1] - Hard Mode (1300ms, 1100ms, 900ms, 700ms, 500ms)

Push Buttons - Directional user inputs.

BTNU - Up Input

BTND - Down Input

BTNL - Left Input

BTNR - Right Input

BTNC - Center Input

1.2.3 Outputs

LED[15:0] - Real-time visual game state indicators. LED[15:0] turns on when the user wins the game and when the correct button is pressed before the time runs out.

7-Segment Display (SEG, AN, DP) - Displays user reaction time by counting up in milliseconds.

UART_TX - Serial output stream utilizing PuTTY to display mode, time, and result of each round played by the user.

- PASS: the correct button is pressed within the required time, the round is considered successful
- LOSE: the wrong button is pressed, the button is pressed too early, or the response time exceeds the limit, the round is considered a failure.
- WIN: the user successfully pressed the correct button within the required time ten rounds in a row.

VGA Interface (Hsync, Vsync, RGB signals) - Generates graphical user interface that highlights the active target direction, and changes the background color depending on the current gameplay condition.

Yellow Screen - Waiting State

Displayed during the WAIT_STATE, before the reaction prompt becomes active. This indicates that the player must wait and avoid pressing any buttons prematurely.

Green Screen - GO State

Displayed during the GO_STATE, when the player is expected to respond to the highlighted directional prompt. The active target arrow is visually emphasized on the screen.

Purple Screen - Result State

Displayed when the player successfully completes the required gameplay. The screen includes a “YOU WIN!” message rendered using VGA pixel graphics.

Red Screen - Lose State

Displayed when the player presses the incorrect button, reacts too early, or exceeds the allowed reaction time. The screen shows a “YOU LOSE!!!” message to indicate a failed attempt.

1.3 FSM Design

The system is controlled by an finite state machine, `game_fsm`, that manages the progression of the reaction time game. This FSM is responsible for controlling game progression, timing, scoring, and response evaluation.

1.3.1 FSM States

IDLE:

- System initialization and reset condition.
- Generates the first random direction using LFSR logic.

WAIT_STATE:

- Implements randomized delay.
- Detects premature button presses and triggers failure state.

GO_STATE:

- Active reaction timing window.
- Evaluates input conditions:
 - Correct input - transition to ROUND_WIN
 - Incorrect input - transition to LOSE_STATE
 - Timeout - transition to LOSE_STATE

ROUND_WIN:

- Correct response evaluation state.
- Increments round counter.
- Initials logging and transition control.

LOSE_STATE:

- Failure handling state.
- Triggers UART logging and visual feedback.

RESULT:

- Final outcome display state (win or lose).

NEXT_ROUND:

- inter-round transition delay.
- Resets timing and prepares the next stimulus.

1.4 FIFO Design

The project incorporates a FIFO buffer in the UARTcommunication subsystem to reliably transmit gameplay results from the FPGA to a computer terminal. The UART logging architecture demonstrates the same sequential storage and retrieval behavior of a FIFO system. The primary implementation of this concept appears in the `uart_logger` module, where a memory array is used to construct and store formatted ASCII messages containing gameplay information such as difficulty mode, reaction time, and win/loss status. During message creation, characters are written sequentially into the array, forming a structured transmission packet. Once logging is initiated, the UART controller reads each character sequentially and transmits the data serially over the UART interface until the terminating null character is reached. This ordered first-in, first-out style of operation mirrors the functional behavior of a traditional FIFO buffer. The UART transmission process itself also demonstrates buffered serial communication behavior. Inside the `uart_tx` module, outgoing parallel byte data is loaded into a 10-bit shift register containing the start bit, eight data bits, and stop bit. The transmitter then shifts the data out bit and a time according to the configured rate timing. This mechanism acts as a temporary bit-level FIFO structure, ensuring that serial data is transmitted in the correct order and at a controlled rate synchronized to the system clock.

Together these buffering mechanisms provide reliable asynchronous communication between the FPGA and the external terminal while maintaining proper timing and data integrity throughout the transmission process.

1.5 Verilog Modeling Style

The project incorporates multiple Verilog modeling styles, including behavioral, dataflow, structural, and gate-level modeling. By combining these techniques, the project achieves a modular and efficient FPGA implementation.

1.5.1 Behavioral Modeling: Behavioral modeling is primarily used for the sequential control logic of the system. This style describes how the hardware behaves over time using procedural blocks such as always @(posedge clk) and conditional state transitions. The most significant use of behavioral modeling appears in the game_fsm module, which implements the finite state machine responsible for controlling gameplay, timing, scoring, and state transitions.

The FSM progresses through states such as IDLE, WAIT_STATE, GO_STATE, ROUND_WIN, LOSE_STATE, and RESULT. Within these states, the system evaluates user input, updates timers, and determines whether the player advances or loses the game. This procedural structure demonstrates how behavioral modeling allows the designer to describe sequential system behavior in a readable and organized manner. Similar behavioral logic is also used in the UART transmitter, VGA timing counters, and seven-segment display refresh circuitry.

1.5.2 Dataflow Modeling: Dataflow modeling is used to describe combinational logic through equations and continuous assignments using the assign statement. This style is heavily used throughout the project for signal generation, button logic, display calculations, and VGA rendering conditions.

For example, the system determines whether any directional button has been pressed and whether the pressed button is incorrect using the following logic:

```
assign any_button = btnU | btnD | btnL | btnR;  
assign wrong_button = any_button && !correct_button;
```

The seven-segment display driver also uses dataflow modeling to separate the reaction time value into individual decimal digits for display:

```
assign ones = number % 10;  
assign tens = (number / 10) % 10;  
assign hundreds = (number / 100) % 10;  
assign thousands = (number / 1000) % 10;
```

Additional dataflow logic is implemented within the VGA controller to define screen regions, arrows, and button highlighting using coordinate comparisons.

1.5.3 Structural Modeling: Structural modeling is used in the top module, where the complete system is assembled by instantiating and connecting smaller submodules together. Rather than describing behavior directly, structural modeling focuses on defining how modules interact through wiring and signal connections. The top module integrates: debounce circuits, game FSM, seven segment display driver, UART logger, and VGA controller.

1.5.4 Gate-Level Modeling: Gate-level modeling is used to directly implement low-level logic gates within the any_button logic circuitry. This modeling style closely resembles actual hardware gate connections and is useful for demonstrating fundamental digital logic implementation. The following code combines all five push-buttons into a single signal named any_button:

```
or g1(t1, btnU, btnD);  
or g2(t2, btnL, btnR);  
or g3(t3, t1, t2);  
or g4(any_button, t3, btnC);
```

The first two OR gates combine pairs of directional buttons. The third OR gate merges those intermediate signals, while the final gate includes the center button. As a result, the output signal any_button becomes high whenever any push-button on the Nexys board is pressed. This gate-level implementation demonstrates direct hardware construction using primitive logic elements and complements the higher-level modeling styles used elsewhere in the design.

1.6 Verification and Testbench Strategy

The system is validated using a unified integration testbench (mother_tb) that simulates the entire hardware architecture simultaneously.

1.6.1 Simulation Structure

The testbench instantiates all major system components to enable full-system validation under realistic operating conditions, including:

- Game FSM
- UART transmitter and logger
- VGA controller
- Top-level integration module

1.6.2 Test Sequence

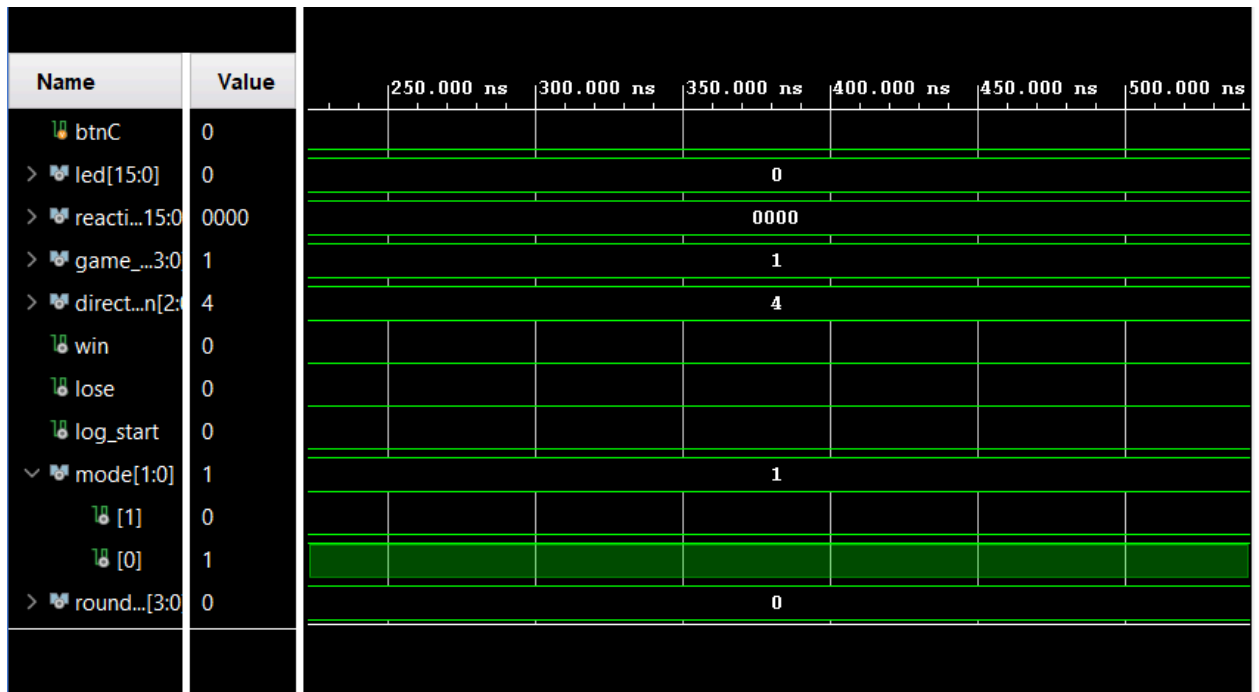
The testbench applies a structured stimulus sequence:

- System reset validation using SW[0]
- Correct button press simulation
- Incorrect button press simulation
- Full top-level system integration test

1.7 Simulation Results

The simulation results shown below verify that the FSM changes states correctly, and responds based on the user's button input.

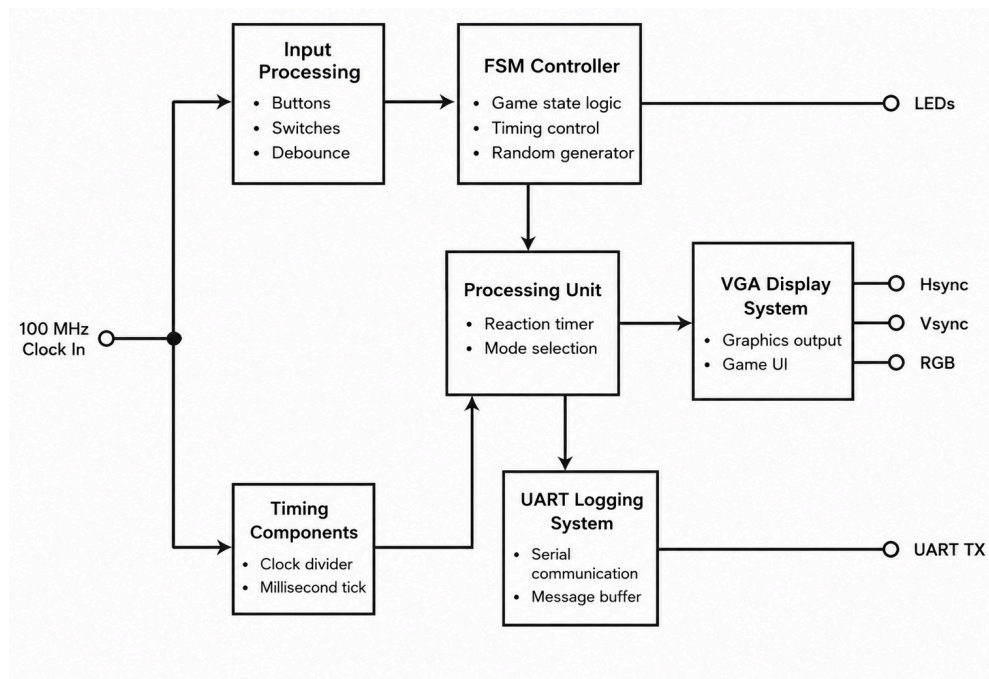
1.7.1



In the simulation, the game starts in the IDLE state. It then gets a start input, which then shifts the state into WAIT_STATE. This causes the LED to stay off while the random delay counter increases. After the wait period, the FSM enters the GO_STATE and the reaction timer starts counting. Upon doing so, the testbench simulates the pressing of a button. When the correct button is pressed, the FSM is shown to enter the round-complete state, show a win, and generate the log_start for starting UART logging. When the wrong button is pressed, it shows a loss but also generates the UART logging signal.

The simulation shows the correct clock signal, FSM moving through states properly, and the reaction timer increasing during the GO_STATE. Additionally, it shows the various types of button inputs (e.g., correct, incorrect, early). Finally, it shows the UART logging signal and that the top modules are connected properly.

1.8. System Block Diagram



1.9 Conclusion

The Reaction Time Game successfully demonstrates a complete FPGA-based embedded system integrating multiple digital design domains. The project combines finite state machine control, real-time timing measurements, graphical VGA rendering, and UART-based communication into a unified architecture.

The design effectively demonstrates the use of structural, dataflow, and behavioral modeling systems within a single system, highlighting the flexibility of Verilog for hardware description. Additionally, the system showcases key embedded design principles such as modular decomposition, synchronous timing control, and multi-interface output coordination.

Overall, the project achieves its goal of implementing an interactive reaction-based game while serving as a comprehensive demonstration of FPGA system design and digital logic integration.